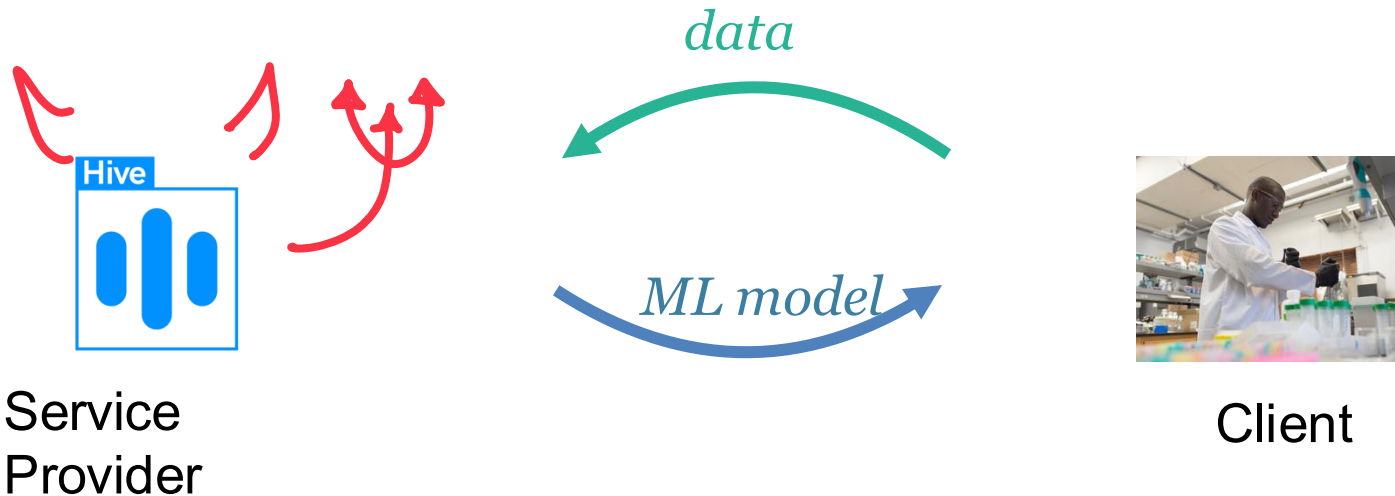


Lecture 9

Verification

Machine Learning as a Service



MLaaS: Amazon
SageMaker/AWS,
Microsoft Azure,
Startups...

Verifying Model Properties, how?



Can we verify properties of the model h :

Accuracy/

Correctness/

Robustness/

Fairness

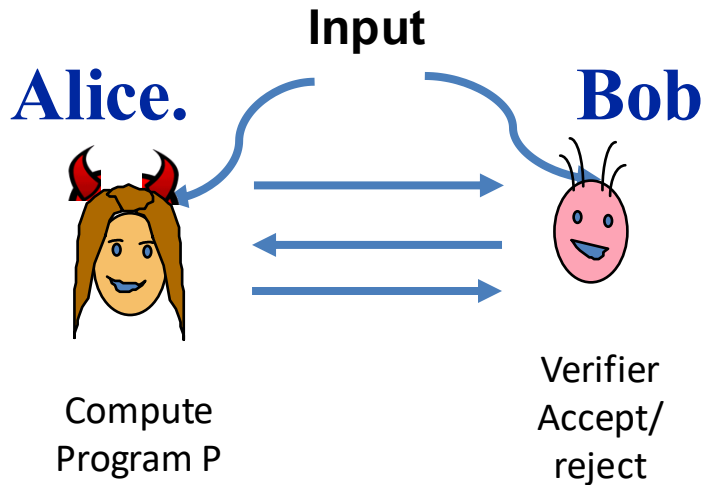
Safety

Satisfies Regulations

cheaply (**not** retraining)
using

- Fewer data samples
- Lower quality data?
- Efficient
Time/Memory
- Black box access to h

1980's Theory on Verifiable Computing

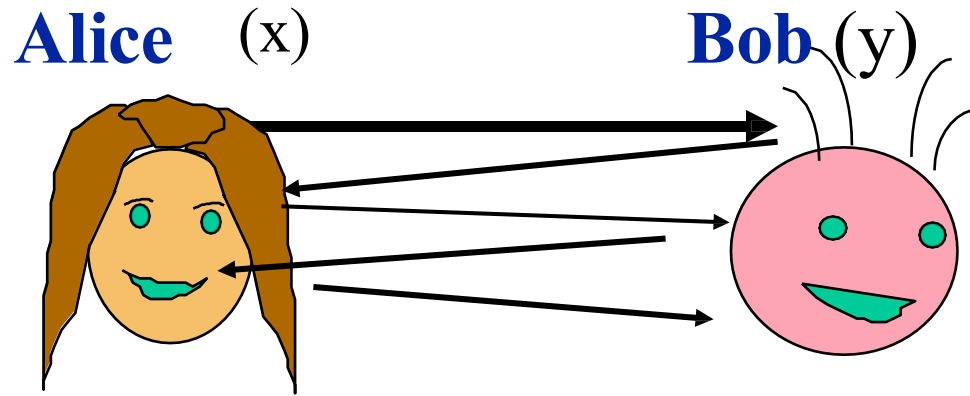


Application to verifiable computing

Many Models & Techniques

- **Interactive Proofs** & arguments for Program Delegation
- **Zero Knowledge** Interactive Proofs & arguments
- **Non-Interactive** Zero Knowledge Proofs & arguments, SNARKs, SNARGs,
- **Multi Prover Proofs**
- **Debates**
- **Probabilistically Checkable Proofs**

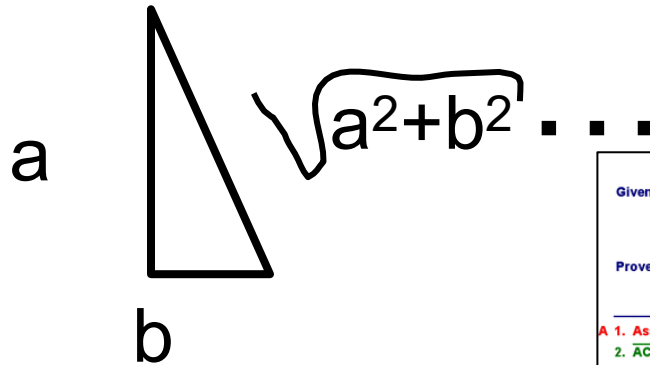
From Secure Communication to Complex Interactions



Now doing much more than communicating securely:

- **Complex interactions:** games, computations, **proofs**
- **Complex Adversaries:** Alice or Bob, adaptively chosen
- **Complex Properties:** correctness, simultaneity, fairness
- **Joined by others:** auctions, bidding, elections, e-commerce

Classical Proofs

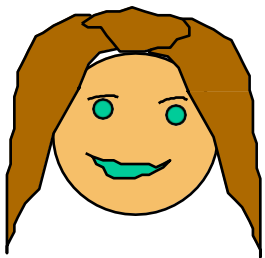


Given: $\overline{AC} \perp \overline{BD}$ $\overline{BC} \equiv \overline{EC}$ $\overline{AB}$ is not $\equiv$ to $\overline{ED}$ Prove: $\angle B$ is not $\equiv$ to $\angle CED$	
Statements	Reasons
A 1. Assume: $\angle B \equiv \angle CED$ 2. $\overline{AC} \perp \overline{BD}$ 3. $\angle BCA$ and $\angle DCE$ are right $\angle$s A 4. $\angle BCA \equiv \angle ECD$ S 5. $\overline{BC} \equiv \overline{EC}$ 6. $\triangle BCA \equiv \triangle ECD$ 7. $\overline{AB} \equiv \overline{ED}$ 8. $\overline{AB}$ is not $\equiv$ to $\overline{ED}$	1. Assumption 2. Given 3. Defn. of $\perp$ segs 4. RAT 5. Given 6. ASA (1, 5, 4) 7. CPCTC 8. Given
But statement 7 contradicts statement 8. Consequently, the assumption must be false. $\therefore \angle B$ is not $\equiv$ to $\angle CED$	

Prime-
Number Thm

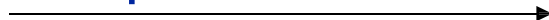
Proofs

Prover



Claim

proof



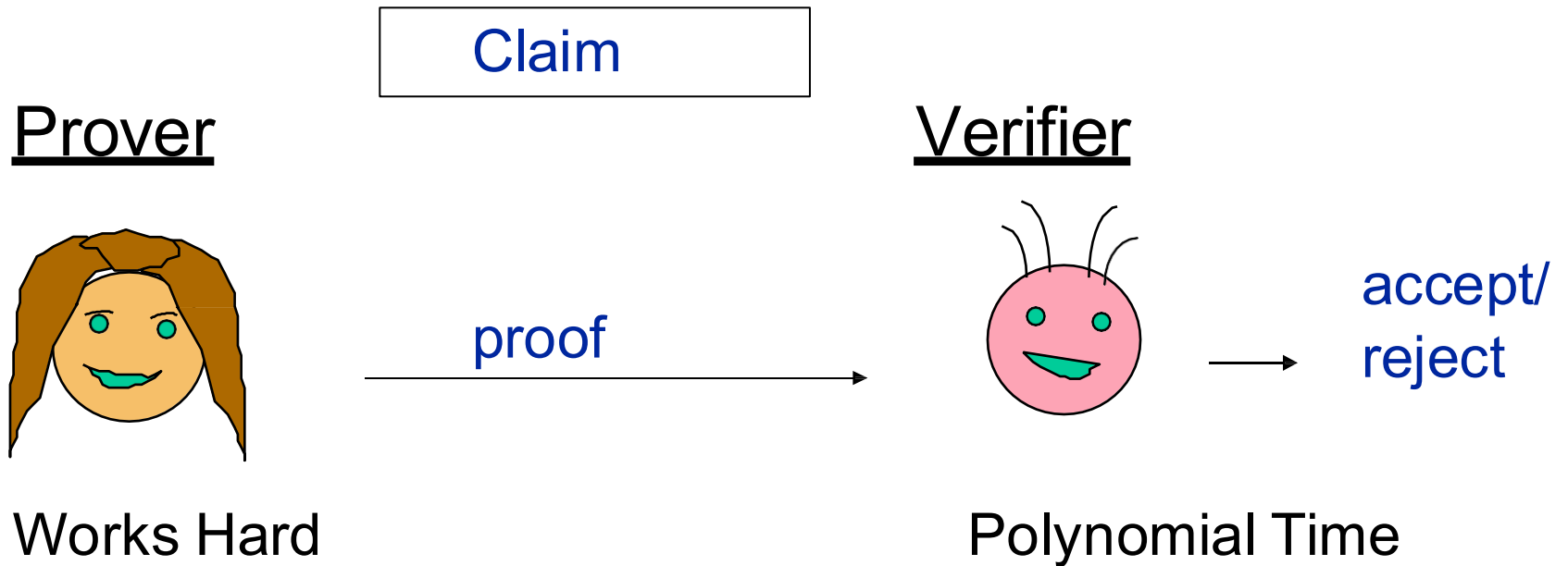
Verifier



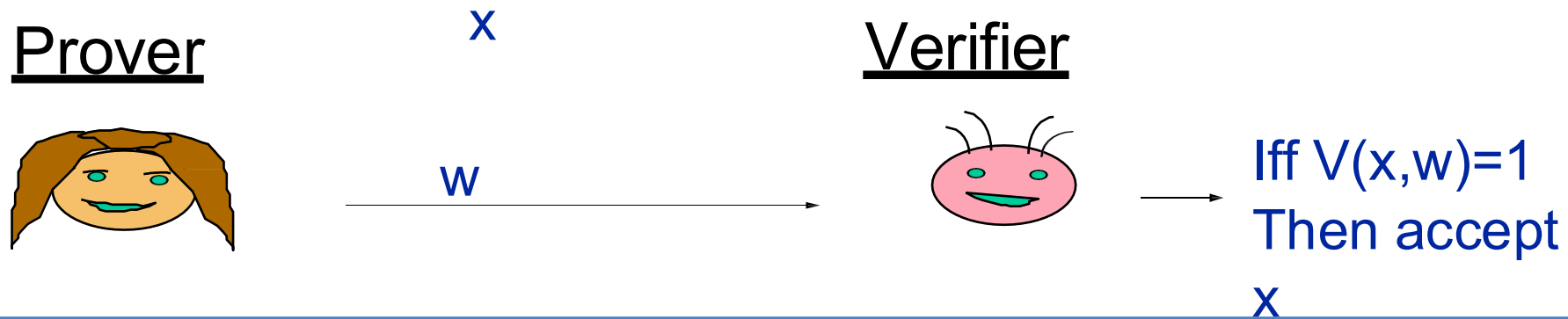
accept/
reject



Efficiently Verifiable Proofs (NP)



Efficiently Verifiable Proofs: $\mathcal{NP}$



Def: D is an $\mathcal{NP}$ -language if there is a **poly-time** verifier V function such that

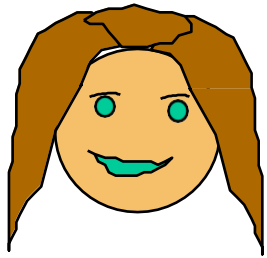
- **Completeness:**

for all $x \in D$, there is a **poly($|x|$)-long** witness (proof) $w \in \{0,1\}^*$ s.t. $V(x, w) = 1$.

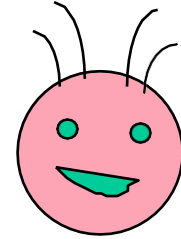
- **Soundness:**

for all $x \notin D$, there is no witness. That is, for all polynomially long $w \in \{0,1\}^*$, $V(x, w) = 0$.

Example: N is a product of 2 large primes



p, q

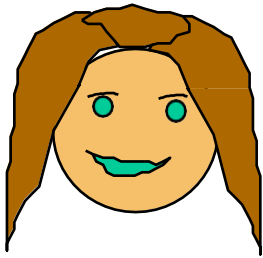


If $N=pq$ and p & q are prime,
then accept
Else reject

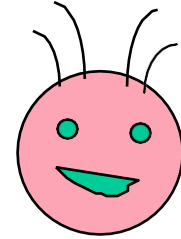
After interaction, Bob knows:

- 1) N is product of 2 primes
- 2) **Also** the factors of N

Example: y is a quadratic residue mod N
(i.e $y = x^2 \pmod{N}$)



x

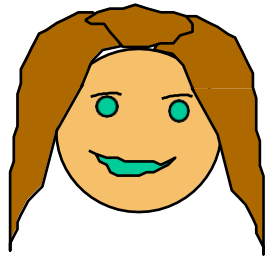
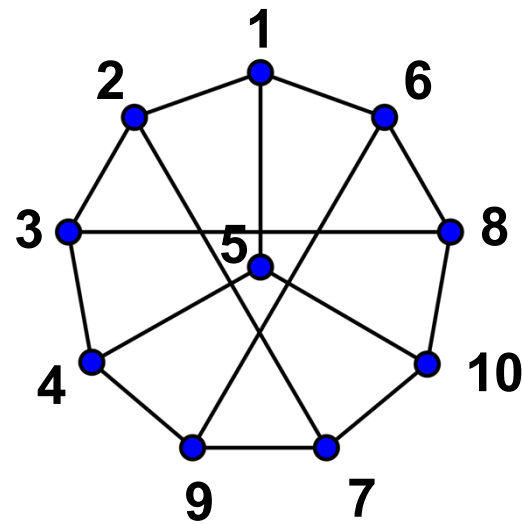
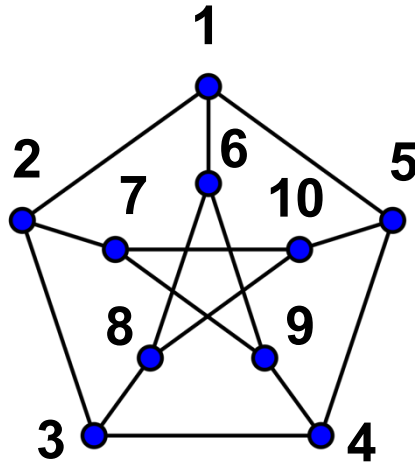


If $y = x^2 \pmod{N}$,
Accept
Else reject

After interaction, Bob knows:

- 1) y is a quadratic residue mod
- 2) Also a square root of $y \pmod{N}$

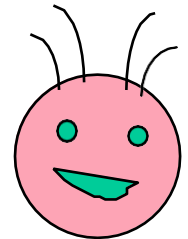
Theorem: Graphs G_0 and G_1 are isomorphic.



Prover

Proof = $\pi: [N] \rightarrow [N]$,

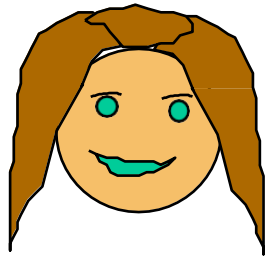
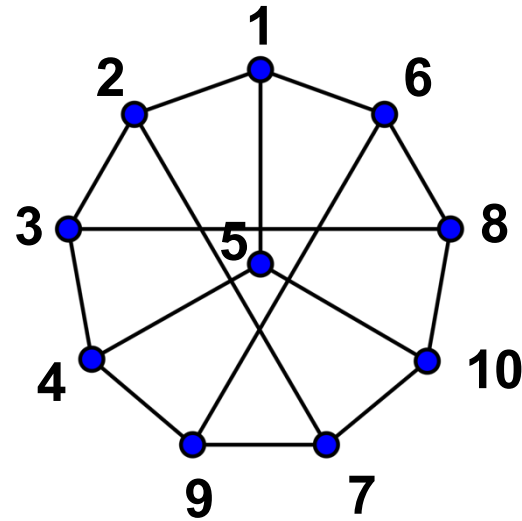
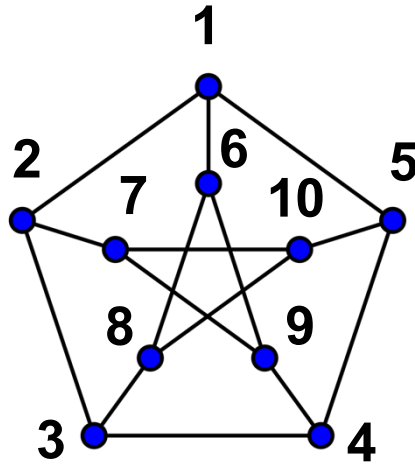
the isomorphism



Verifier

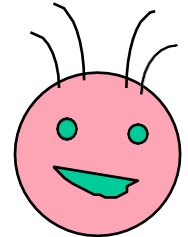
Check $\forall i, j$:
 $(\pi(i), \pi(j)) \in E_1$ iff $(i, j) \in E_0$.

Theorem: Graphs G_0 and G_1 are isomorphic.



Prover

π

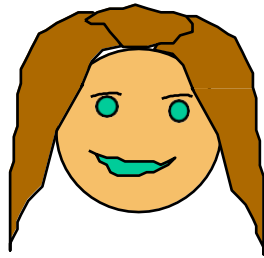
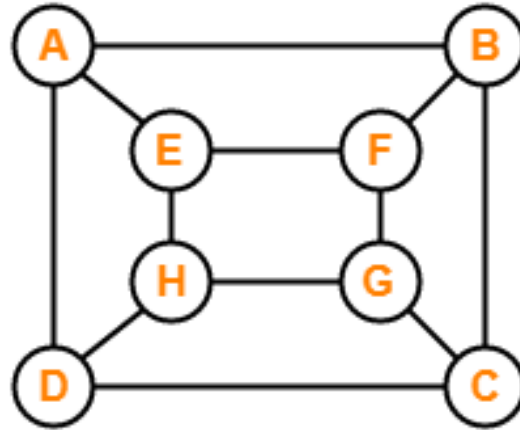


Verifier

After interaction, Bob knows:

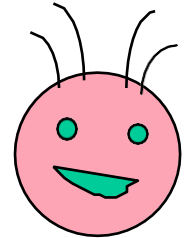
- 1) G_0 is isomorphic to G_1
- 2) **Also** the isomorphism π

Theorem: Graphs G has a Hamiltonian cycle.



Prover

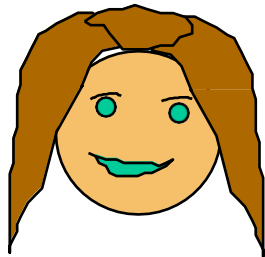
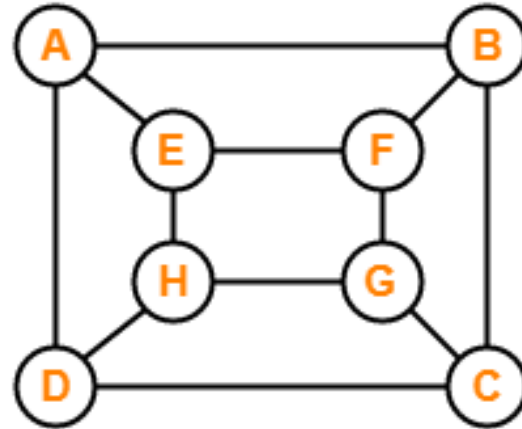
**Proof = Hamiltonian
cycle**
 $(v_0, \dots, v_{N-1})$



Verifier

Check $\forall i$:
 $(v_i, v_{i+1 \bmod N}) \in E$

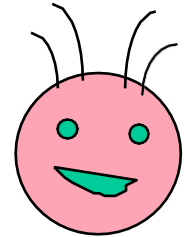
Theorem: Graphs G has a Hamiltonian cycle.



Prover

Hamiltonian cycle

$(v_1, \dots, v_N)$

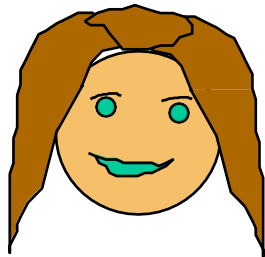
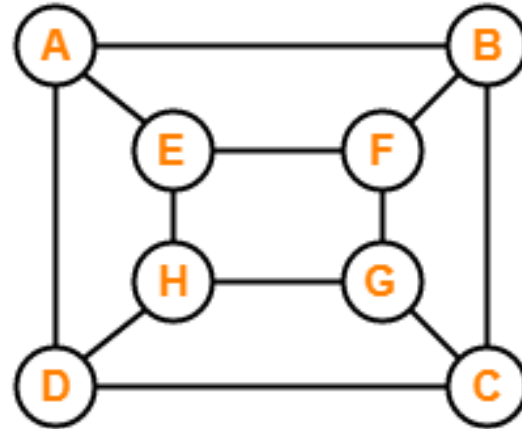


Verifier

After interaction, Bob the Verifier knows:

- 1) G **has** a Hamiltonian cycle.
- 2) **Also**, the Hamiltonian cycle itself.

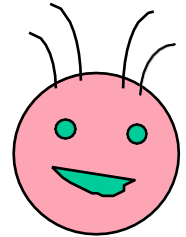
Theorem: Graphs G has a Hamiltonian cycle.



Prover

Hamiltonian cycle

$(v_1, \dots, v_N)$

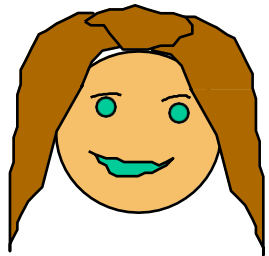


Verifier

NP-Complete Problem:

Every one of the other problems can be reduced to it

Is there any other way?



I will not give you
an isomorphism, but I will prove
to you that I could provide one.

HOW?

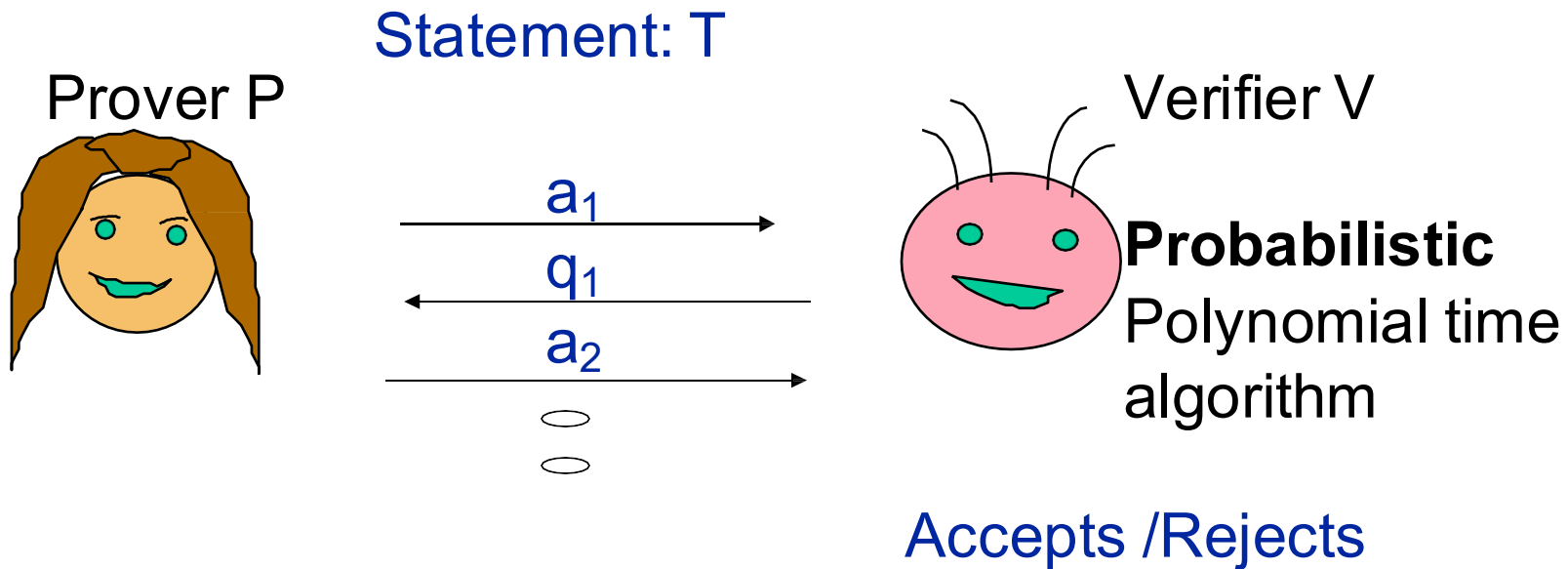
Two New Ingredients

Interactive and Probabilistic Proofs

Non-trivial interaction: rather than “reading” proof, verifier engages in a non-trivial **interaction** with the **prover**.

Randomness: verifier is randomized (tosses coins as a primitive operation), and can err with some small probability

Interactive Proofs[GMR85]

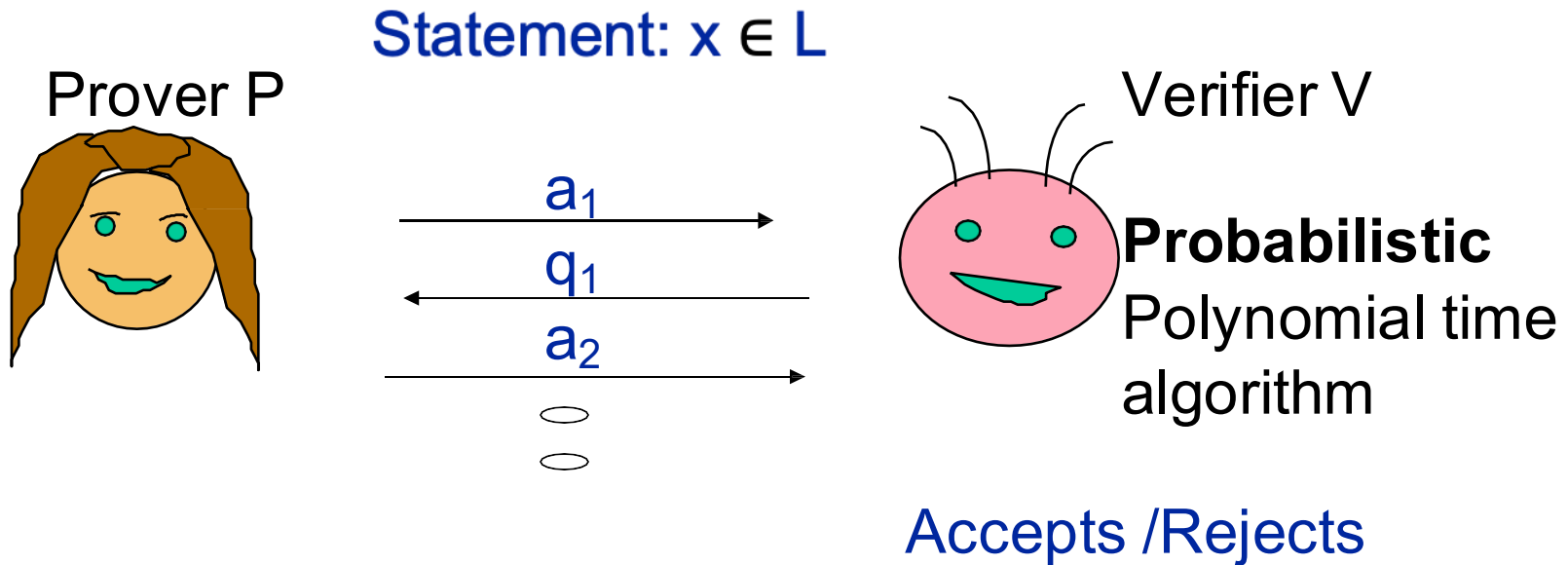


(P, V) is an **interactive proof system for T** if

Completeness: if T is true, then V will always accept

Soundness: if T is false, then regardless of prover P^* strategy, V will reject with overwhelming probability

Interactive Proofs for Language L[GMR85]



(P, V) is an **interactive proof system** for L if

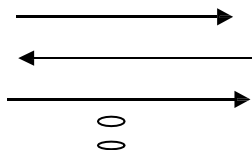
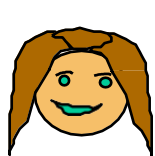
Completeness: if $x \in L$, then $\text{Prob}[(P, V)[x] = \text{accept}] = 1$

Soundness: if $x \notin L$, then $\forall P^*$
 $\text{Prob}[(P^*, V)[x] = \text{accept}] = \text{neg}(|x|)$

Remarks: Interactive Proofs

Prover P

Verifier V



Probabilistic
Polynomial time

Accepts
/Rejects

P and V are a pair of **interactive Algorithms**, each having private inputs and private coins as well as a common public input.

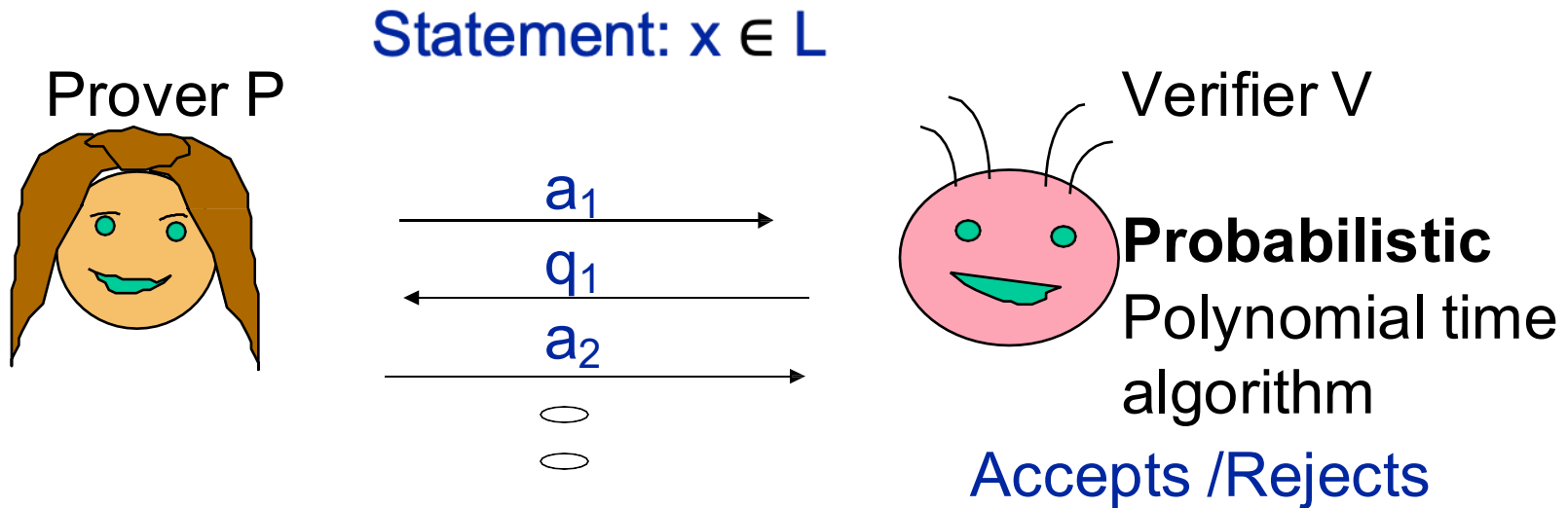
V additionally must run in polynomial time

(P,V) satisfy completeness $c(x)$ & soundness $s(x)$

- if $x \in L$, $\text{Prob}((P,V)[x] = \text{accepts}) > c(x)$
- If $x \notin L$, $\forall P^*$, $\text{Prob}[(P^*,V)[x] = \text{accepts}] < s(x)$

Suffice to require: $c(x) = 2/3$ and $s(x) = 1/3$

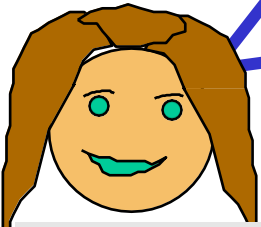
Class IP



IP = $\{L \text{ s.t. there exists } (P, V) \text{ interactive proof system for } L$
with completeness $c(x)=2/3$
and soundness $s(x)=1/3\}$

Can you prove more with coins and interactions?
Is IP greater than NP?

The Idea Graph ISO



I will produce a random graph H for which

1: I can give you an isomorphism γ_0 from G_0 to H

OR

2: I can give you an isomorphism γ_1 from G_1 to H

Proof:

$$H = \gamma_0(G_0),$$

$$H = \gamma_1(G_1),$$

Thus

$$G_1 = \gamma_1^{-1}(\gamma_0(G_0))$$

$$\text{Set } \sigma = \gamma_1^{-1} \gamma_0$$

Hence, there is an isomorphism σ from G_0 to G_1 directly

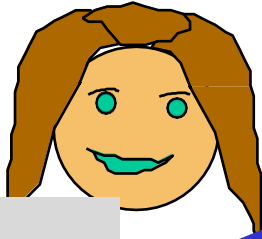
YOU randomly choose if I should demonstrate my ability to do

#1 or **#2**.

POINT IS: If I can do both, there exists an isomorphism from G_0 to G_1

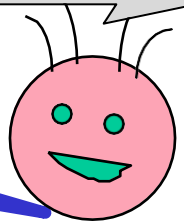
REPEAT K
INDEPENDENT
TIMES.

An Interactive Proof



Graph H

Toss
coin b



b

Choose
random γ_0
permutation
of vertices
of G_0 . Set
 $H = \gamma_0(G_0)$

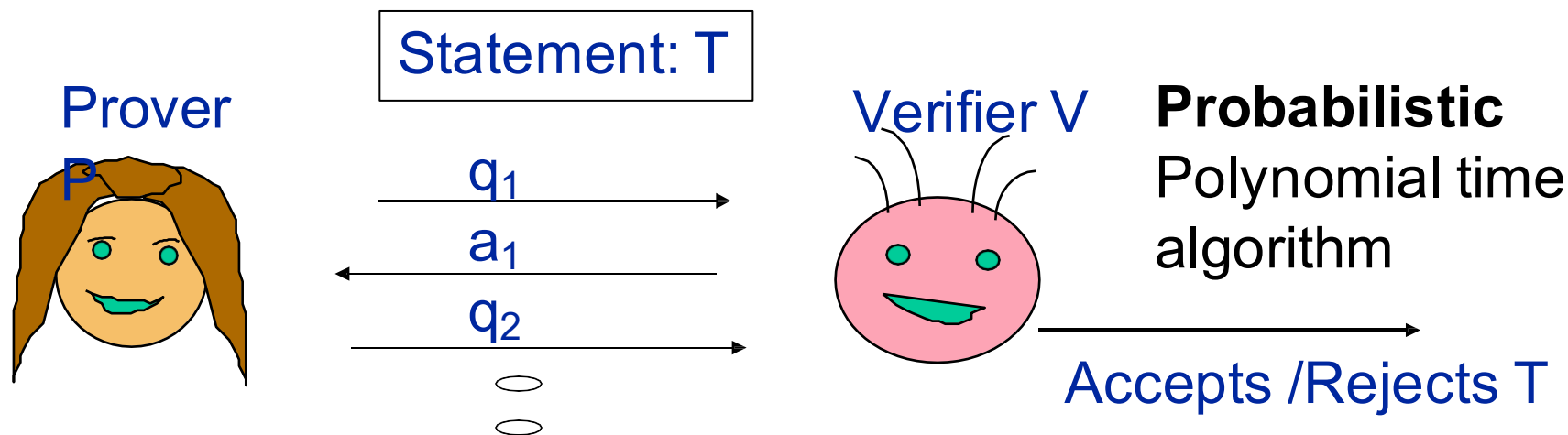
If $b=0$: send γ_0

If $b=1$: send $\gamma_0 \sigma^{-1}$ (where $\sigma(G_0)=G_1$)

Claims:

- (1) Statement true $\rightarrow$ can answer correctly for $b=0$ and 1
- (2) Statement false $\rightarrow \text{prob}_b(\text{catch a mistake}) = 1 - 1/2^k$
- (3) Zero Knowledge (to be defined)

Zero Knowledge Interactive Proofs



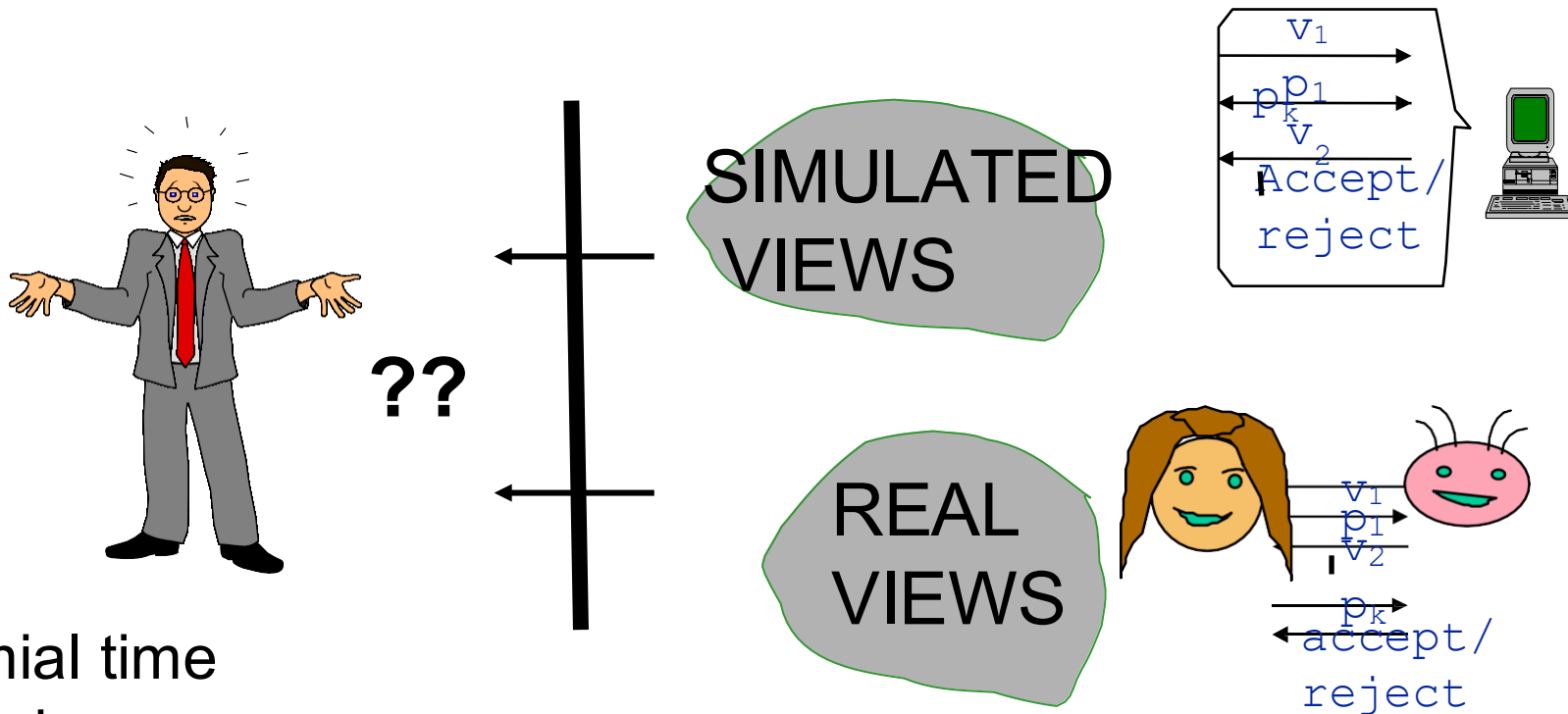
After interactive proof, V “knows”:

- T is true (or $x \in L$)
- A **view** of interaction (=transcript + coins V tossed)

P gives Zero- Knowledge to V: when T is true, the **view** gives V nothing he couldn't have obtained on his own without interacting

How Do we Capture Getting “Nothing Extra”(when T is true)

If: the verifier's view can be efficiently simulated so that `simulated views' and `real views' are indistinguishable in polynomial time.



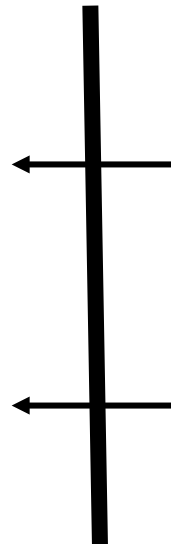
Polynomial time
distinguisher

Perfect Zero Knowledge (when T is true)

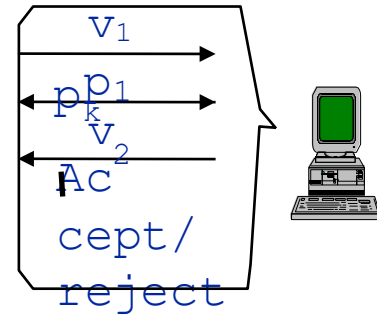
If: the verifier's view can be efficiently simulated
so that 'Simulated views' = 'real views'



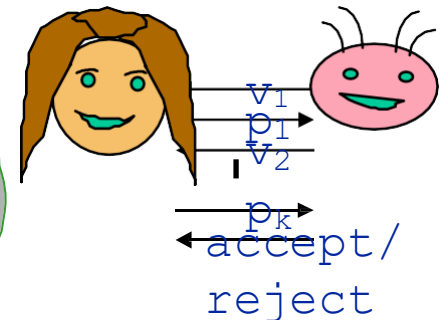
??



SIMULATED
VIEWS



REAL
VIEWS



The distinguisher
Any Algorithm

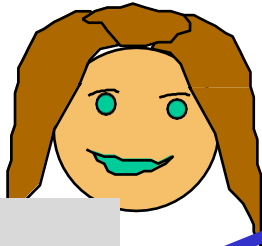
Formal Definition: Perfect Zero-Knowledge

For a given P and V on input x , define probability space $\text{View}_{(P,V)}(x) = \{(q_1, a_1, q_2, a_2, \dots, \text{coins of } V)\}$ (over coins of V and P)

(P, V) is **honest verifier perfect zero-knowledge** for L if:
 $\exists$ SIM a polynomial time randomized algorithm s.t. $\forall x$ in L , $\text{View}_{(P,V)}(x) = \text{SIM}(x)$

Will allow SIM
Expected polynomial
time

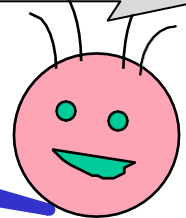
Recall: Isomorphism Example



Graph H

Toss
coin b

b



Choose
random γ_0
permutation
of vertices
of G_0 . Set
 $H = \gamma_0(G_0)$

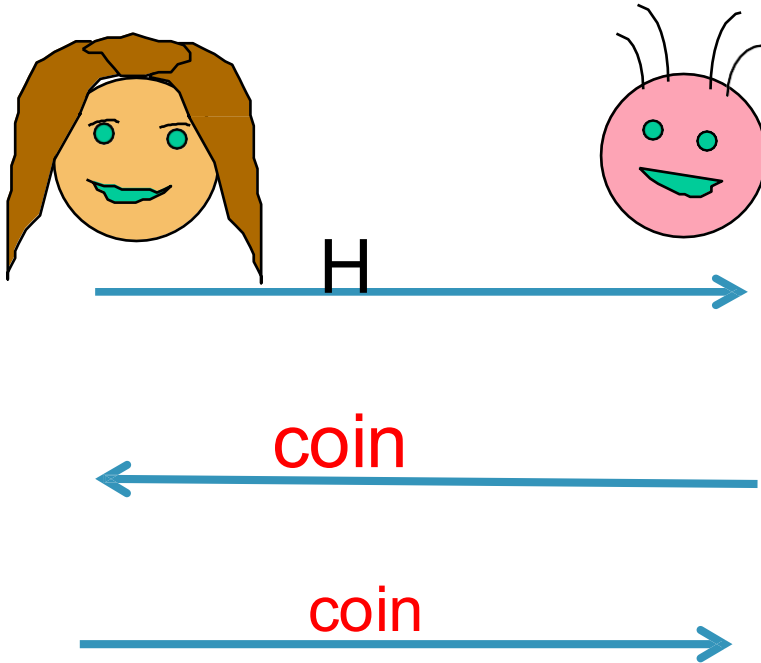
If $b=0$: send γ_0

If $b=1$: send $\gamma_0 \sigma^{-1}$ (where $\sigma(G_0) = G_1$)

View of Bob =

$\{(H; b; \text{random isomorphism from } G_b \text{ to } H)\}$

Zero Knowledge



SIMULATOR M:

- toss **coin**
- If **coin**=head:
choose random γ_0
set $H = \gamma_0(G_0)$
- If **coin**=tail choose
random γ_1
set $H = \gamma_1(G_1)$

View of Bob=

$\{(H, \text{coin}, \text{random isomorphism of } G_b \text{ to } H)\}$

What if V is not honest:

Perfect Zero-Knowledge (Final def)

For a given P and V on input x, define probability space $\text{View}_{(P,V)}(x) = \{(q_1, a_1, q_2, a_2, \dots, \text{coins})\}$ (over coins of V and P)

(P,V) is honest verifier perfect zero-knowledge for L if:

$\exists$ SIM expected polynomial time randomized algorithm s.t. $\forall x$ in L, $\text{View}_{(P,V)}(x) = \text{SIM}(x)$

(P,V) is perfect zero-knowledge for L if :

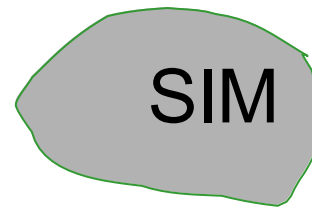
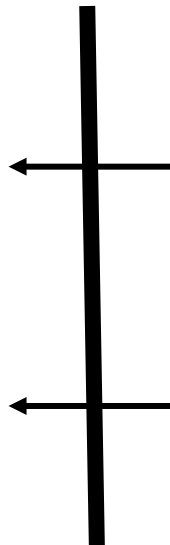
$\exists$ SIM expected polynomial time randomized algorithm s.t. $\forall \text{PPT } V^*$ $\forall x$ in L, $\text{View}_{(P,V^*)}(x) = \text{SIM}(x)$

Prover Gives Perfect Zero Knowledge

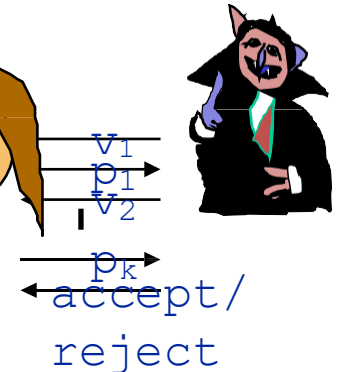
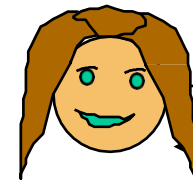
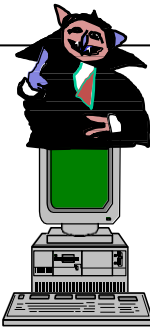
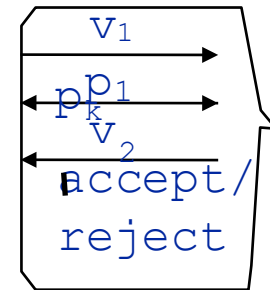
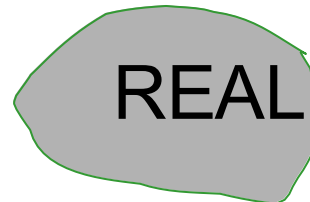
- If: we can efficiently simulate the view of any verifier s.t. 'Simulated views' = 'real verifier' for any poly time verifier



??

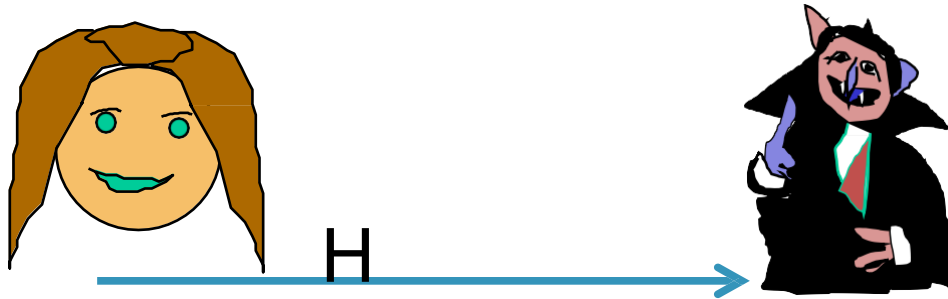


||



The observer
Any Algorithm

Zero Knowledge Proof that G_1 isomorphic to G_2



← coin

if coin=coin. answer
Else abort and try again →

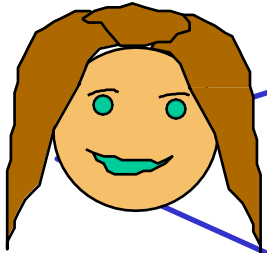
SIMULATOR SIM:

1. toss coin
2. If coin=head:
choose random y_0
set $H = \gamma_0(G_0)$
If coin=tail
choose random γ_1
set $H = \gamma_1(G_2)$
3. Feed H to V^*
4. If V^* outputs 
coin==coin
output $(H, \text{coin}, \gamma_{\text{coin}})$
Else abort and
goto 1 again.

Claim:

$\text{prob}[\text{coin}=\text{coin}] = \frac{1}{2},$
Expected [number of repetitions of SIM] = 2.
For k repetitions, SIM expected trials = $2k$

Claim: $y = x^2 \pmod N$ is solvable



Choose
 $1 < r < n$ at
random

Repeat 100 times

$$z = [r^2 \pmod n]$$

$$zy = [(rx)^2 \pmod n]$$

- If I gave you solutions to both, that is r and rx , you would be convinced that the claim is true but also know x
- Instead, I will give you a solution to only one equation, either r or rx but you can choose which!

$$1 - \left(\frac{1}{2}\right)^{100}$$

Flip a



to choose an equation

Gives a solution to the equation
requested

Accepts claim
only if gets
correct
solution



Zero Knowledge Proof that $Y=x^2 \bmod N$



if **coin** $\neq$ **coin** abort
If **coin**=**coin**, send r

SIMULATOR SIM:

1. toss **coin**

2. If **coin**=**head**:
choose random r
set $z=r^2 \bmod n$

If **coin**=**tail**

choose random r

set $z=(ry^{-1})^2 \bmod n$

3. Feed z to $V^* =$ 

4. If $V^*(z)$ outputs **coin** $\neq$ **coin**

abort and goto 1

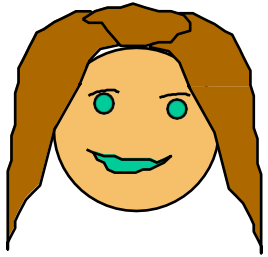
else for **coin**=**head**

output(H, coin, r) &

for **coin**=**tail**,

output(H, coin, r)

Zero Knowledge Proof that $Y=x^2 \bmod N$



z



← coin →

if coin $\neq$ coin abort
If coin=coin, send r →

Claim:

$\text{prob}[\text{coin}=\text{coin}] = \frac{1}{2},$

Expected [number of repetitions of M] = 2. For k repetitions. M expected trials = 2k

SIMULATOR SIM:

1. toss coin

2. If coin=head:
choose random r
set $z=r^2 \bmod n$

If coin=tail

choose random r

set $z=(ry^{-1})^2 \bmod n$

3. Feed z to $V^* =$



4. If $V^*(z)$ outputs coin $\neq$ coin
abort and goto 1

else for coin=head
output(H, coin, r) &
for coin=tail,
output t(H, coin, r)

SIM: Expected Polynomial Time

- Analysis can be confusing
- Instead can change def to allow
 - $\text{SIM}(x)$ to output $\perp$ with probability at most $1/2$ and require
 - $\text{View}(x) = \text{SIM}(x)$ to be conditioned on the event that $M(x)$ does not output $\perp$
 - $1/2$ can be relaxed to $\text{neg}(x)$

What Made it possible?

Randomness

- The statement to be proven has **many possible proofs** of which the prover chooses one ***at random***.
- Each such proof is made up of exactly 2 parts: seeing either part on its own gives the verifier no knowledge; seeing both parts imply 100% correctness.
- Verifier chooses **at random** which of the two parts of the proof he wants the prover to give him. The ability of the prover to provide either part, convinces the verifier

Actually, Alice seems to have proved more: that she actually “knows” the isomorphism (square root)

Let V be polynomial time relation. Let $(x, w) \in V$
 V defines Language $L_V = \{x \mid \exists w \text{ s.t. } V(x, w) = 1\}$.

We say that (P, V) is a **proof of knowledge** for L_V

[or that P on x knows w] *if*:

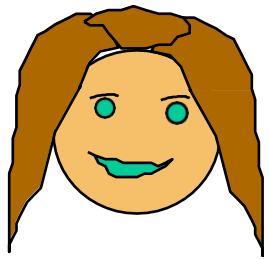
$\exists$ an **extractor** algorithm E s.t. for all x

$E^P(x)$ outputs w *in expected polynomial time*

ZKPOK: zero knowledge proof of knowledge y
a

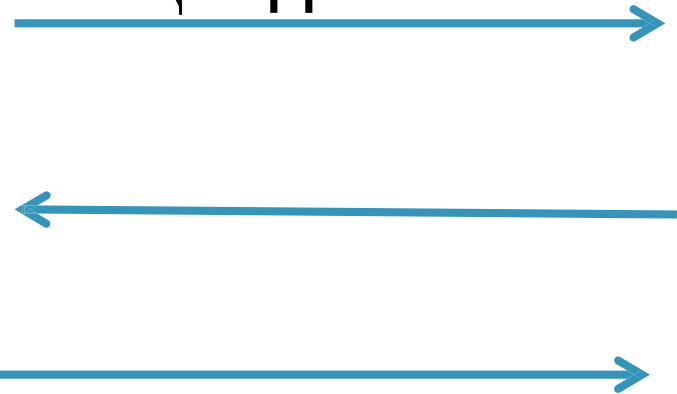
This is called the **rewinding technique**

ZKPOK that Prover knows an isomorphism from G_1 to G_2



H

Extractor
Algorithm



Extractor  :

- 1) On input H
set **coin**=head
Store γ_0
- 2) **Rewind** and 2nd time
set **coin**=tail
Store γ_1
- 3) Output $\gamma_1^{-1}(\gamma_0)$

Computational Zero-Knowledge

(P, V) is **honest verifier perfect zero-knowledge** for L if:
 $\exists$ SIM an expected polynomial time randomized algorithm s.t. $\forall x$ in L , $\text{View}_{(P, V)}(x) \approx_c \text{SIM}(x)$

Relax to “indistinguishable” by any observer who runs
In probabilistic polynomial time



(P, V) is **computational zero-knowledge** for L if : **$\forall \text{PPT } V^*$**
 $\exists$ SIM an expected polynomial time randomized algorithm s.t. $\forall x$ in L , $\text{View}_{(P, V^*)}(x) \approx_c \text{SIM}(x)$

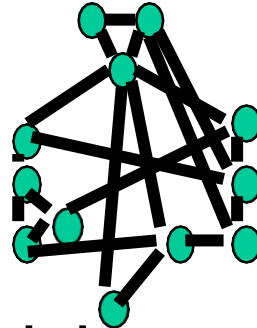
Notation: $\text{View}_{V^*}(x) \approx_c \text{SIM}(x)$

Zero Knowledge for all of NP

Theorem: If one-way functions exist, then every problem in NP has a computational zero knowledge interactive proofs

- To prove the theorem, should we construct ZK proof for every **NP** language?
- Enough to show zero knowledge interactive proof for one NP complete problem

Why is that enough?



3COLOR = all graphs which can be colored with 3 colors
s.t for for all edges (u,v) $\text{color}(u) \neq \text{color}(v)$

NP Completeness [Cook-Levin-Karp]: Given L in NP.

Instances x is polynomial time reducible to G_x

$x \in L \quad \longrightarrow \quad G_x \text{ is 3 colorable}$

$x \notin L \quad \longrightarrow \quad G_x \text{ is not 3 colorable}$

Show a
Zero-knowledge
Proof for 3-coloring

Intuition for Protocol

On common input graph $G=(V,E)$ and

Provers private input coloring $\alpha: V \rightarrow \{\textcolor{red}{0}, \textcolor{blue}{1}, \textcolor{green}{2}\}$

- P picks a random permutation π of the colors $\{\textcolor{red}{0}, \textcolor{blue}{1}, \textcolor{green}{2}\}$ & color the graph with coloring $\sigma = \pi(\alpha)$. It hides the color $\sigma(u)$ of each vertex inside a **locked box**
- V Select a random edge (u,v)
- P opens **boxes** corresponding to u and v
- V accepts if and only if $\sigma(u) \neq \sigma(v)$
[colors are different]

Completeness and Soundness

- **Completeness:** if prover uses a proper 3-coloring, the verifier will accept.

- **Soundness:** Let $k = |E|^2$

If G is not 3-colorable, then for all P^*

$$\text{Prob}[(P^*, V)(G) \text{ accepts}] < 1 - 1/|E|$$

Repeat k times.

Soundness $\text{Prob}[(P^*, V)(G) \text{ accepts}] <$

$$(1 - 1/|E|)^k < 1/e^{|E|}$$

Commitment Scheme:

Digital analogue of locked boxes

- An efficient two-stage protocol between a sender and receiver on security parameter 1^k s.t.:
- **commit stage:** sender has private input b
At the end of the commit stage
 - both parties hold output **com** (called the commitment)
 - sender holds a private output **dec** (called the de-commitment)
- **reveal stage:** sender sends the pair (dec', b') to receiver.
Receiver accepts or rejects $(\text{com}, \text{dec}, b)$

Hiding and Binding Properties

Commitment schemes satisfy two properties:

- **Hiding:** Given **comm** sent by sender, receiver should not be able to b and b' for $b \neq b'$
- **Binding:** *Sender* can not produce two tuples $(comm, dec, b)$ and $(comm, dec, b')$ that the receiver will accept both with high probability if $b \neq b'$.

Example: use $Enc(r, b)$ for commitment to b

ZK interactive proof for G3COL

On common input graph $G=(V,E)$ and private prover input coloring $\alpha: V \rightarrow \{0,1,2\}$

- $P \rightarrow V$: Pick a random permutation π of the coloring & color the graph with coloring $\pi(\alpha(v))$. Send commitments $Enc(r_v, \pi(\alpha(v))) \forall$ vertex v .
- $V \rightarrow P$: Select a random edge (u,v) and send it
- $P \rightarrow V$: reveal colors of u and v committed to by releasing r_u and r_v
- If *color $u = color v$* , **V rejects**, otherwise repeat
- **V accepts** after k iterations.

Simulation for any Verifier V^*

Simulator SIM on input $G=(V,E)$ and verifier

V^* : Fix random tape ω for V^*

For $i = 1$ to $|E|^2$:

- Choose random edge (a, b) and permutation π of the colors, generate vector $\text{com} = \text{Enc}(r_v, \alpha(\pi(v)))$ as in honest verifier simulation.
- Run $V^*(\text{com}; \omega)$ to obtain challenge (a^*, b^*) ; if $(a^*, b^*) = (a, b)$, then output
 $\text{transcript} = (\text{Enc}(r_v, \alpha(\pi(v))) \forall v; (a, b); r_a, r_b)$

If all iterations fail, output $\perp$.

Theorem: If Enc is computationally secure with respect to non-uniform adversaries, then

Claim 1: $\forall G, \alpha$ (a true coloring) : $\text{prob}[\perp \text{ output}] = \text{neg}(|E|)$

Claim 2 :if $\perp$ is not output, then $\text{simulated-view} \approx_c \text{real-view}$